CoRT: Code-integrated Reasoning within Thinking

Taro Yamada[1]

[1]Anonymous Institution

Abstract

Large reasoning models are strong at long-form mathematical reasoning, yet they remain unreliable at exact computation and often waste tokens by invoking tools too late or re-checking deterministic outputs. We present CoRT, a post-training framework for code-integrated reasoning that teaches models to interleave natural-language thought with Python execution inside a persistent environment. CoRT uses minimal but targeted hints to trigger code at computation bottlenecks, trust execution results, and reduce redundant verification, then strengthens these behaviors through a staged pipeline of cold-start prompting, rejection fine-tuning, strong-to-weak distillation, and code-aware reinforcement learning. On five competition-math benchmarks, the 32B CoRT models reach 81.8 average accuracy for Prompt-Hint-SFT and 81.3 for Hint-Engineering-RFT, outperforming the DeepSeek-R1-32B baseline by about 4 points on average while remaining substantially more token efficient. On lightweight models, Prompt-Hint-1.5B-RL reaches 58.3 average accuracy, improving over its SFT counterpart by 7.8 points and surpassing DeepScaleR-1.5B-Preview. Behavioral analyses show that hint engineering changes tool use from predominantly verification-oriented to a more balanced mix of calculation and verification, while RL and code-aware rewards improve low-sample efficiency and peak AIME24 accuracy.

1 Introduction

Large language models increasingly solve difficult mathematical tasks by producing long chains of thought, a capability sharpened by supervised fine-tuning and reinforcement learning [1–4]. However, long reasoning traces alone do not eliminate a basic failure mode: symbolic and arithmetic subproblems still require exact computation, and even strong reasoning models remain brittle when they must manually manipulate expressions, enumerate cases, or verify tedious derivations. External tools help with those subproblems [5–8], but tool integration introduces a second challenge. The model must decide *when* to call code, *what* to send to the executor, and *how* to use the result without collapsing into either under-use or wasteful over-verification.

Recent tool-integrated reasoning systems show that code execution can materially improve mathematical reasoning [7, 9, 10]. Yet current approaches still face two practical bottlenecks. First, high-quality traces that interleave natural language and code are scarce. Second, even when models can call a code interpreter, they often do so at the wrong time: either too late, after already spending thousands of tokens on fragile hand calculations, or too often, by repeatedly re-checking already-correct Python outputs. In both cases, the result is lower efficiency and weaker sample efficiency.

We study a simple thesis: for tool-integrated reasoning, a small amount of *behaviorally targeted supervision* can matter more than large amounts of generic tool traces. CoRT operationalizes this thesis with minimal prompts and hint-engineered edits that nudge the model toward two behaviors: call Python early when a subproblem becomes tedious or exact, and trust deterministic execution once the computation has succeeded. The framework then consolidates those behaviors through

Preprint.

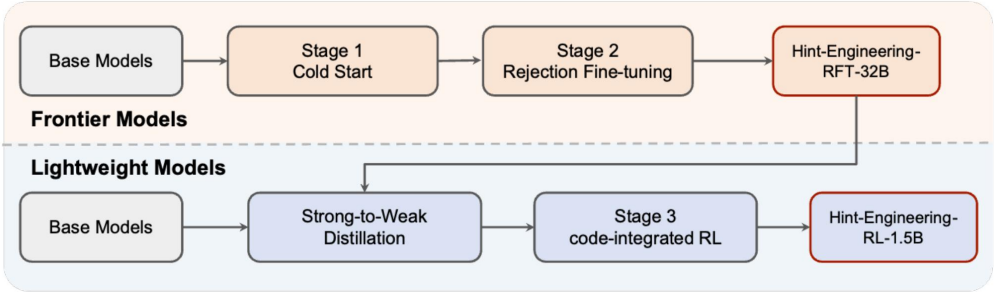


Figure 1: CoRT training pipeline. Frontier 32B models are improved through cold-start hinting and rejection fine-tuning. The resulting strong model is then distilled into a 1.5B student, which is further improved by code-integrated RL.

staged post-training, including rejection fine-tuning for 32B models and strong-to-weak distillation followed by GRPO-style RL for 1.5B models.

Our contributions are threefold. First, we introduce a hint-based data synthesis framework for code-integrated mathematical reasoning that starts from a small curated seed set and expands into SFT and RFT data. Second, we implement a code-aware rollout pipeline with iterative prompt-code-execute loops, persistent execution context, masking of executor outputs during policy optimization, and an auxiliary code reward. Third, we show consistent accuracy and efficiency gains across AIME24, AIME25, AMC23, MATH500, and OlympiadBench, together with analyses that expose how the learned tool-use behavior changes.

2 Related Work

Long chain-of-thought reasoning. Chain-of-thought prompting and its successors established that explicit intermediate reasoning can substantially improve mathematical performance [1, 11]. More recent reasoning models scale this idea with longer traces, curated reasoning data, and outcome-based RL [2–4, 12, 13]. These models are strong but often pay for accuracy with long outputs, and they still rely heavily on internal computation even when exact execution would be cheaper and more reliable.

Tool use for reasoning. Tool augmentation has been studied in both general and math-specific settings. Toolformer learns API usage from self-supervision [8]; PAL and Program-of-Thoughts separate reasoning from computation by delegating program execution to an external runtime [5, 6]. In mathematical reasoning, ToRA demonstrates the value of interleaving text and tools, and CRITIC emphasizes iterative verification with external feedback [7, 14]. More recent reasoning-tool hybrids such as START and ReTool show that hinting and RL can improve tool-using LRMs [9, 10]. Our work is closest in spirit to START's hint-based stimulation, but differs in focusing on minimal targeted hints as a data engine for full post-training, plus explicit efficiency mechanisms during RL.

Data-efficient post-training for reasoning. A separate line of work argues that reasoning can emerge from surprisingly small, carefully selected datasets [15, 13]. ReFT, iterative reasoning preference optimization, and process-supervision methods show that how reasoning traces are filtered and optimized matters as much as raw scale [16–19]. CoRT follows this line by using a small curated seed set to induce tool-using behaviors, then reinforcing them with rejection sampling, distillation, and RL. Unlike process-reward approaches, our method remains outcome-driven but augments the training loop with tool-specific inductive bias.

3 CoRT Method

3.1 Problem Setting

We consider mathematical reasoning with access to a Python code interpreter. Given a problem x, the model generates a trace inside a `<think>` block that may interleave natural language and

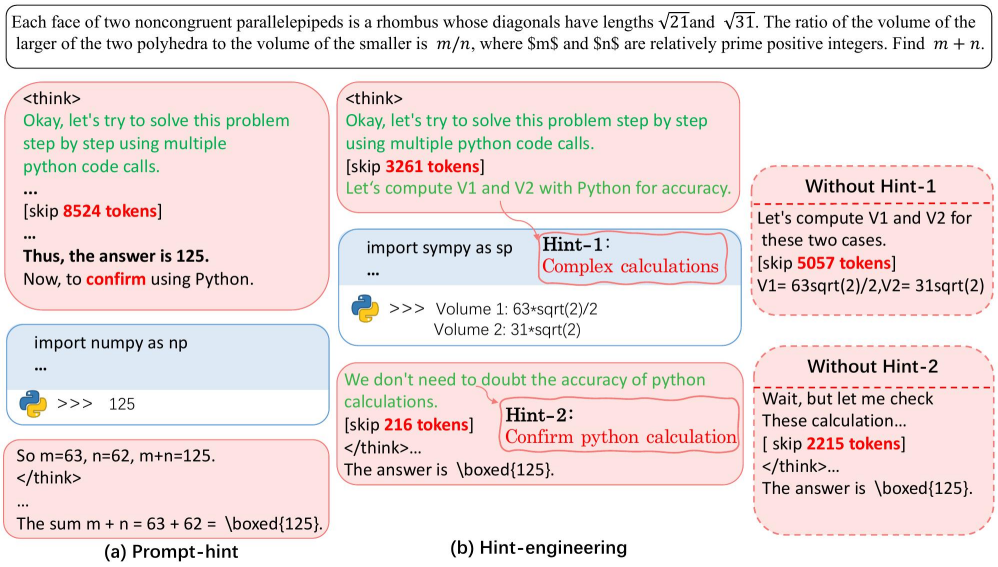


Figure 2: Case study comparing Prompt-Hint and Hint-Engineering on AIME23-I Problem 13. Prompt-Hint delays tool use and spends most of its budget on verification. Hint-Engineering inserts local cues that trigger computation earlier and reduce redundant post-execution checking.

fenced Python code. Whenever a Python block is produced, an executor runs the code and appends a corresponding ‘‘‘`output ...` ‘‘‘ block to the context. Generation then continues from the augmented context. The central challenge is not only correctness of the final boxed answer, but also learning an efficient policy for *where* tool calls belong inside the reasoning process.

In the released inference and rollout pipeline, this loop is implemented as an iterative prompt-code-execute process with a persistent Jupyter-like interaction pattern, capped tool budgets, and multi-sample evaluation. The runtime executes generated Python with timeouts, appends outputs back to the model context, and continues generation until termination or the tool limit is reached. For evaluation we use up to 15 tool calls and a maximum sequence length of 32,768 tokens.

3.2 Prompt Hints and Hint Engineering

The first ingredient in CoRT is a *general prompt hint* placed immediately after `<think>`, e.g.,

```
Okay, let's try to solve this problem step by step using multiple
python code calls.
```

This prompt-hint is deliberately lightweight: it does not prescribe a full reasoning template, but makes code use salient early enough that the model is more likely to call Python before entering lengthy hand computation.

The second ingredient is *targeted hint engineering*. Here, we edit trajectories with short cues inserted at behaviorally important moments. Two recurring cases are: (i) the reasoning reaches a tedious exact-computation step, where we insert a cue such as “It looks tedious, and we can use python code”; and (ii) the model starts doubting a correct execution result, where we insert a cue such as “We don’t need to doubt the accuracy of python calculations.” These edits explicitly bias the model toward earlier code triggering and lower post-execution hesitation.

Figure 2 shows a representative example from AIME23-I Problem 13. Prompt-Hint eventually solves the problem correctly, but only calls Python after more than 8k tokens and then uses the interpreter to confirm an answer it had effectively already obtained. Hint-Engineering triggers code much earlier, reaches the decisive computation after roughly 3.3k tokens, and then terminates with only a short post-execution continuation instead of a second verification loop.

3.3 Data Construction and Staged Post-Training

CoRT uses a staged training pipeline summarized in Figure 1. We begin from a small curated seed set of 30 high-quality AIME traces. Using prompt hints and hint-engineered editing, we synthesize a larger set of code-integrated traces from 820 auxiliary problems sourced from STILL-3-style mathematical data. This process yields 800 SFT examples and 830 RFT examples in the experimental record accompanying this paper.

The frontier-model branch targets 32B models. Stage 1 uses cold-start hinting to elicit code-interleaved trajectories from the base model. Stage 2 applies rejection fine-tuning, selecting and refining generated traces to form Hint-Engineering-RFT-32B. This branch is designed to maximize tool competence without RL.

The lightweight-model branch targets 1.5B models. We first distill a strong 32B teacher into a smaller student, transferring both answer quality and tool-use format. We then apply Stage 3 code-integrated RL, which continues optimization in the same prompt-code-execute environment used at inference time. This strong-to-weak route is important because naïvely training a small model to discover good tool policy from scratch is substantially harder than refining a student that already imitates coherent tool traces.

3.4 Code-Aware RL

Our RL stage uses GRPO-style outcome optimization, following recent reasoning RL systems [20, 3, 12]. The reward combines answer correctness with a small auxiliary tool term:

$$R = R_a + \omega R_c, \qquad \omega = 0.1.$$

Here R_a is the task reward derived from the final boxed answer, while R_c reflects code-use quality. In the released implementation, this auxiliary term appears as a small ± 0.1 adjustment based on execution quality: degenerate traces that only produce erroneous outputs are penalized, and an optional LLM-based scorer can add or subtract a small amount depending on whether Python was used appropriately.

Two implementation details are important. First, RL rollouts occur in the same iterative execution loop as inference, with capped numbers of function calls and accumulated execution outputs. Second, CoRT masks executor-produced ```output ... ``` spans during policy optimization. This prevents the actor from spending probability mass learning to reproduce deterministic interpreter outputs, and instead focuses the update on decisions the model actually controls: when to call code, how to write it, and how to continue reasoning after seeing the result. In practice, this masking is implemented by identifying output blocks in decoded responses and zeroing their contribution in the response mask used by the actor loss.

4 Experimental Setup

4.1 Benchmarks and Metrics

We evaluate on five mathematical reasoning benchmarks: AIME24, AIME25, AMC23, MATH500, and OlympiadBench [21, 22]. The main metric is accuracy. For AIME24, AIME25, and AMC23, results are computed over 16 samples per problem; for MATH500 and OlympiadBench, we use 4 samples. All evaluations use temperature 0.6, top-p 0.95, a maximum context length of 32,768 tokens, and at most 15 tool calls, matching the provided evaluation setup. In addition to final accuracy, we analyze pass@k, token usage, and tool-use behavior categories.

4.2 Baselines

We compare against strong no-tool and tool-using baselines. At the high end these include o1, DeepSeek-R1, and QwQ-32B [2, 23]; among open 32B frontier models we compare to DeepSeek-R1-32B, START-32B, STILL-3-TOOL-32B, and ReTool-R1-32B [4, 9, 10]; and for lightweight models we compare to DeepSeek-R1-1.5B, DeepScaleR-1.5B-Preview, and ToRL-1.5B [24]. These baselines span both pure long-CoT reasoning and prior tool-integrated post-training approaches.

4.3 Implementation Notes

The inference code uses a Python executor with timeouts and safe execution restrictions, majority-style multi-sample evaluation, and answer extraction based on boxed expressions plus symbolic verification. The RL recipe uses GRPO with KL regularization, dynamic batch sizing, and vLLM-based rollouts. The released training script sets 8 rollout samples during training, 16 during validation, response lengths up to 16k tokens for training and 31k for validation, and 12/15 maximum tool calls for train/validation respectively.

5 Results

5.1 Main Benchmark Results

Table 1: Performance comparison of different math reasoning models across benchmarks. For each section, best results are shown in **bold** and second-best results are underlined. During inference, we set temperature 0.6 and top_p 0.95. Results for AIME24, AIME25, and AMC23 are averaged over 16 samples, while MATH500 and Olympiad results are averaged over 4 samples. All experiments use a maximum sequence length of 32,768 tokens and limit tool usage to 15 calls.

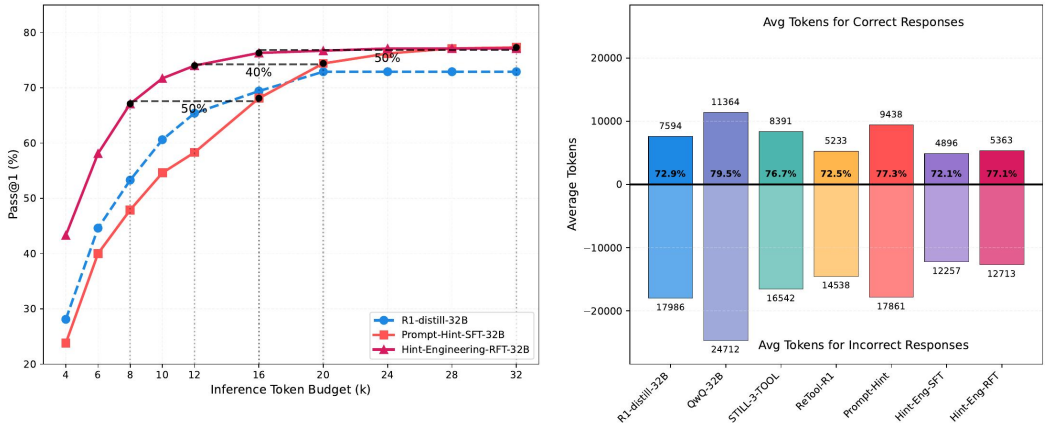
Model	Tool-Use	AIME24	AIME25	AMC23	MATH500	Olympiad	Avg
SOTA Models							
o1	✗	74.3	**79.2**	-	96.4	-	-
DeepSeek-R1	✗	**79.8**	70.0	-	**97.3**	-	-
QwQ-32B	✗	79.5	65.3	**94.3**	92.3	**79.7**	**82.2**
Frontier Models (32B)							
DeepSeek-R1-32B	✗	72.9	59.0	88.8	94.3	72.5	77.5
START-32B	✓	66.7	47.1	**95.0**	94.4	-	-
STILL-3-TOOL-32B	✓	76.7	64.4	91.3	**96.6**	**75.9**	81.0
ReTool-R1-32B	✓	72.5	54.3	92.9	94.3	69.2	76.6
Prompt-Hint-SFT-32B	✓	**77.3**	65.0	**95.0**	**96.6**	75.1	**81.8**
Hint-Engineering-SFT-32B	✓	72.1	60.2	91.3	94.4	71.2	77.8
Hint-Engineering-RFT-32B	✓	76.7	**67.1**	94.4	95.1	73.4	81.3
Lightweight Models (1.5B)							
DeepSeek-R1-1.5B	✗	28.8	21.8	62.9	83.9	43.3	48.1
DeepScaleR-1.5B-Preview	✗	40.0	30.0	73.6	**87.8**	50.0	56.3
ToRL-1.5B	✓	26.7	26.7	67.5	77.8	44.0	48.5
Prompt-Hint-1.5B-SFT	✓	30.6	25.0	63.1	83.3	50.4	50.5
Prompt-Hint-1.5B-RL	✓	**43.1**	**30.2**	**73.8**	87.3	**57.1**	**58.3**
Hint-Engineering-1.5B-SFT	✓	34.0	23.5	64.6	84.2	49.8	51.2
Hint-Engineering-1.5B-RL	✓	41.0	29.4	70.0	85.8	55.6	56.4

Table 1 shows that CoRT improves both 32B and 1.5B models. Among tool-using 32B models, Prompt-Hint-SFT-32B obtains the best average score at 81.8, while Hint-Engineering-RFT-32B reaches 81.3 and achieves the strongest AIME25 score at 67.1. Relative to DeepSeek-R1-32B, the best CoRT variant improves the five-benchmark average from 77.5 to 81.8. These gains are not universal: QwQ-32B still has the highest overall average among all listed 32B systems and remains stronger on OlympiadBench, while DeepSeek-R1 remains best on some individual non-32B entries. The point of CoRT is therefore not that tool use dominates every benchmark, but that carefully trained tool use yields a stronger efficiency-accuracy tradeoff than either text-only reasoning or naïve tool invocation.

The lightweight results are also strong. Prompt-Hint-1.5B-RL reaches 58.3 average accuracy, improving over Prompt-Hint-1.5B-SFT by 7.8 points and surpassing DeepScaleR-1.5B-Preview by 2.0 points. Hint-Engineering-1.5B-RL reaches 56.4, which is lower than Prompt-Hint-1.5B-RL in average accuracy but remains competitive while offering better efficiency in later analyses.

5.2 Token Efficiency

Figures 3 and 4 show that CoRT improves efficiency as well as accuracy. On AIME24, Hint-Engineering-RFT-32B reaches roughly the same accuracy band as Prompt-Hint-SFT-32B and STILL-



(a) Performance on AIME24 with different token budget

(b) Average token usage on AIME24

Figure 3: Token-efficiency analysis on AIME24. Left: pass@1 as a function of the inference token budget. Right: average token usage for correct and incorrect responses. Hint-Engineering-RFT-32B reaches competitive AIME24 accuracy with substantially fewer tokens than comparable 32B baselines.

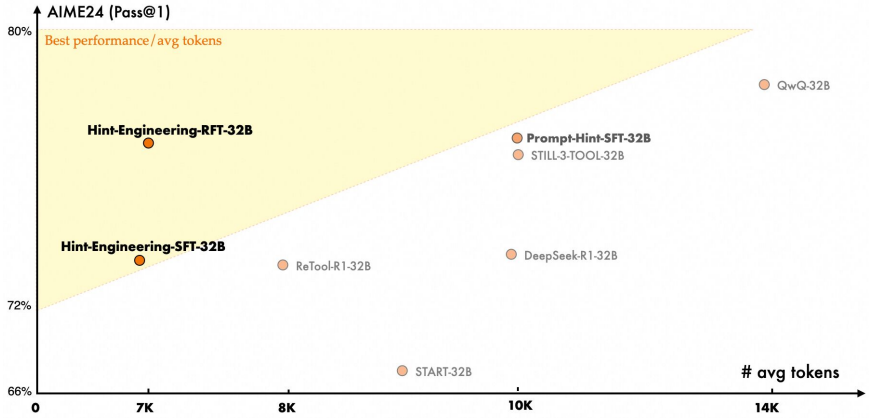


Figure 4: AIME24 pass@1 versus average tokens. The upper-left region is better. Hint-Engineering-RFT-32B occupies the best accuracy-efficiency frontier among the shown 32B models.

3-TOOL-32B while using substantially fewer tokens, and it is far more efficient than QwQ-32B. The summary scatter places Hint-Engineering-RFT-32B near 77% pass@1 at about 7k average tokens, whereas QwQ-32B reaches about 79.5% at roughly 14k tokens. The detailed token plot further shows that Hint-Engineering-RFT-32B uses about 5.4k average tokens on correct responses, versus about 11.4k for QwQ-32B. Prompt-Hint-SFT-32B is more accurate than Hint-Engineering-RFT-32B on the full average metric, but its AIME24 token footprint is closer to 9.4k average tokens for correct responses. Overall, targeted hint engineering offers a better accuracy-per-token frontier than generic prompt hints alone.

5.3 Behavioral Analysis of Tool Use

Figure 5 clarifies why hint engineering is more efficient. Under Prompt-Hint, 82.4% of code usage is devoted to verification and only 17.6% to direct calculation. Hint-Engineering shifts this split to 48.9% verification and 51.1% calculation. The per-category breakdown shows that Prompt-Hint is dominated by property verification and theorem checking (51.0%), whereas Hint-Engineering spreads code use across solving equations, numerical approximation, symbolic manipulation, modular arithmetic, combinatorial enumeration, and geometry. This is exactly the intended effect of the targeted hints: they convert code from a late-stage checker into a first-class computational primitive.

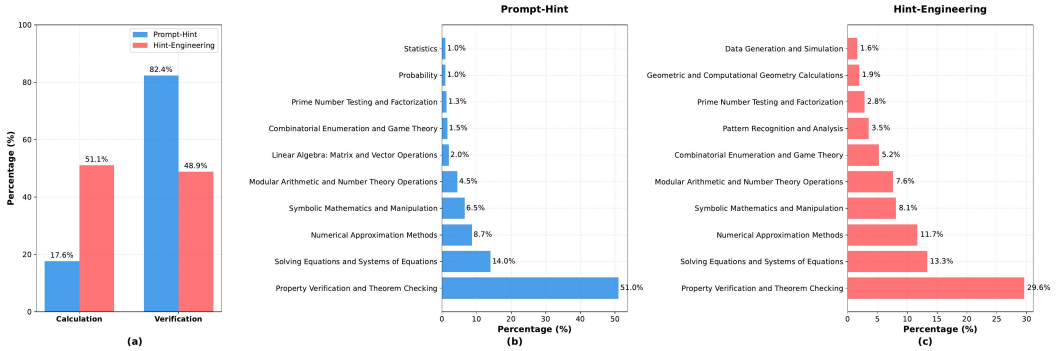


Figure 5: Behavioral analysis of Python usage. Left: Prompt-Hint relies heavily on verification, while Hint-Engineering produces a much more balanced split between calculation and verification. Middle and right: category breakdowns of Python usage for the two approaches.

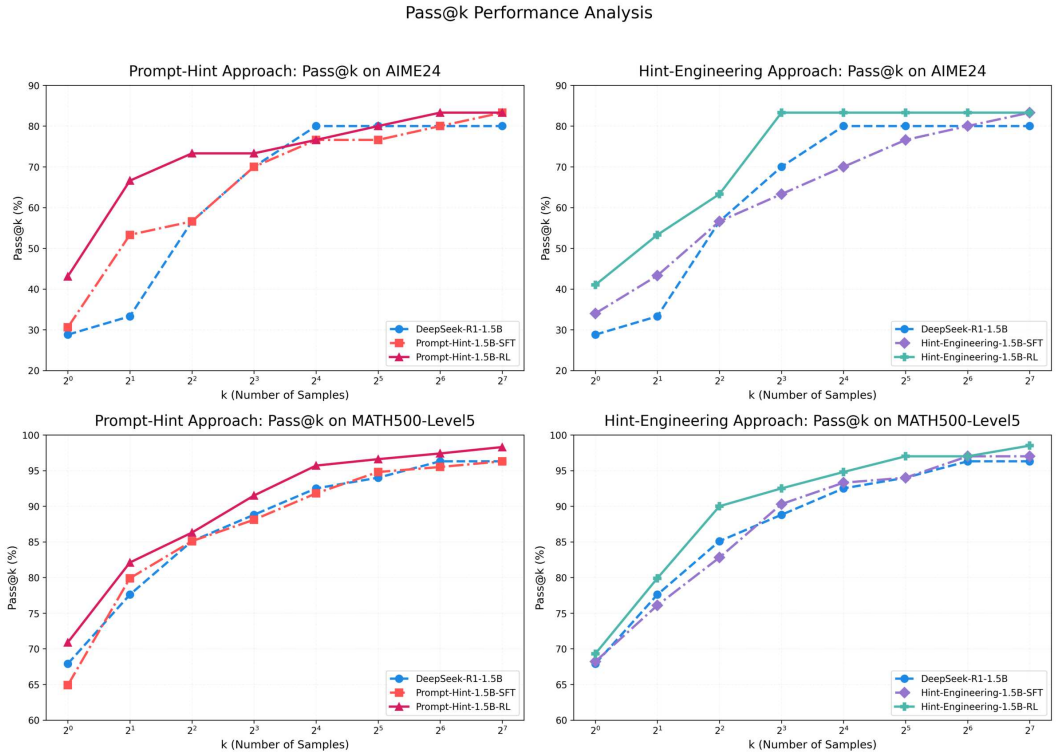


Figure 6: Pass@k curves on AIME24 and MATH500-Level5 for Prompt-Hint and Hint-Engineering at 1.5B scale. RL mainly improves low-k sample efficiency, with especially strong gains on AIME24.

6 Analysis

6.1 Pass@k and Sample Efficiency

Figure 6 shows that RL improves more than the final asymptote. On Prompt-Hint-1.5B, pass@1 on AIME24 rises from about 30.6 for SFT to 43.1 for RL, while the gap narrows at larger k. Hint-Engineering exhibits the same pattern: RL gives markedly better low-k performance and reaches strong pass@k values earlier. On MATH500-Level5, the gains are smaller but still consistent, especially in the low- and mid-k regime. This suggests that code-integrated RL primarily improves policy quality and sample efficiency, not merely diversity over many draws.

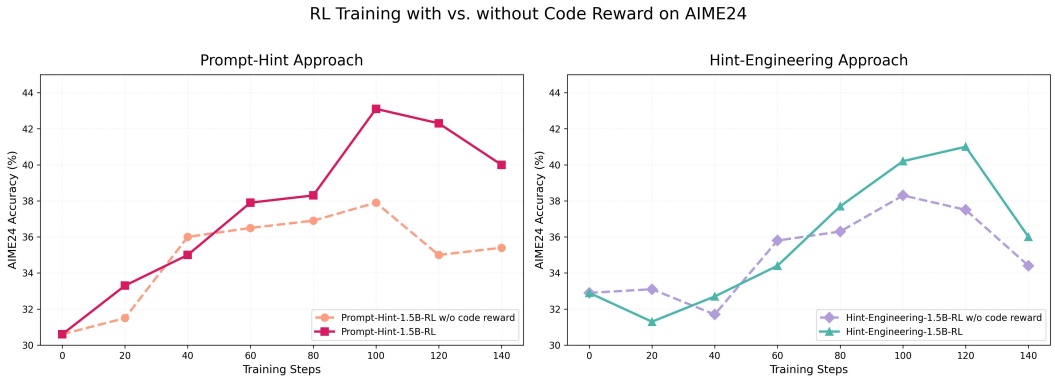


Figure 7: Ablation of the auxiliary code reward during 1.5B RL on AIME24. The reward consistently improves peak performance, with larger gains for Prompt-Hint.

6.2 Effect of Code Reward

Figure 7 ablates the code reward used during RL. For Prompt-Hint-1.5B-RL, adding the reward raises peak AIME24 accuracy from about 37.9 to 43.1, a gain of roughly 5 points. For Hint-Engineering-1.5B-RL, the reward improves the best accuracy from about 38.3 to 41.0. These gains are meaningful given that the auxiliary term is deliberately small. The result supports the claim that the reward is not replacing answer supervision; rather, it gently biases the policy away from pathological tool traces and toward more effective code usage.

Taken together, the pass@k, token, and behavior plots suggest that CoRT changes the *shape* of reasoning rather than only its endpoint. Prompt-Hint mainly raises capability by making tool access available, but Hint-Engineering and RL additionally compress the path to a correct answer: the model reaches decisive computations earlier, spends fewer samples on redundant checking, and converts more of its token budget into useful search. This view is consistent with the strongest gains appearing at low k and moderate token budgets. It also explains why the 1.5B student benefits disproportionately from RL: once the model learns a workable tool policy, better control over *when* to execute code recovers accuracy that would otherwise require a larger parametric reasoner. In other words, CoRT is especially valuable in the practically relevant regime where one cannot afford either very long traces or many independent samples.

6.3 What CoRT Solves and What It Does Not

CoRT addresses a specific failure mode of tool-using LRMs: poor control over *when* code should appear and *how much* reasoning should follow it. The results show that carefully engineered hints and code-aware RL substantially improve this control. At the same time, our results are not evidence that tool use universally dominates text-only reasoning. QwQ-32B remains the strongest average 32B model in Table 1, and DeepSeek-R1 remains stronger on some high-end entries. The more precise conclusion is that CoRT improves the efficiency and reliability of tool-integrated reasoning, particularly when compute budgets or model size are constrained.

7 Conclusion

We presented CoRT, a post-training framework for code-integrated mathematical reasoning built around a simple idea: minimal, behaviorally targeted hints can teach a reasoning model to call tools earlier and trust them more appropriately. Combined with rejection fine-tuning, strong-to-weak distillation, and code-aware RL, this produces both accuracy gains and substantial token savings across five math benchmarks. The analyses show that CoRT changes tool use qualitatively, shifting from verification-heavy behavior to a more balanced reasoning policy that uses code for actual computation. These results suggest that efficient tool use is not only a capability problem, but also a supervision-design problem.

References

[1] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[2] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

[3] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL https://arxiv.org/abs/2503.14476.

[4] Zhipeng Chen, Yingqian Min, Beichen Zhang, Jie Chen, Jinhao Jiang, Daixuan Cheng, Wayne Xin Zhao, Zheng Liu, Xu Miao, Yang Lu, et al. An empirical study on eliciting and improving r1-like reasoning models. *arXiv preprint arXiv:2503.04548*, 2025.

[5] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *International Conference on Machine Learning*, pages 10764–10799. PMLR, 2023.

[6] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.

[7] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. Tora: A tool-integrated reasoning agent for mathematical problem solving. *arXiv preprint arXiv:2309.17452*, 2023.

[8] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL https://arxiv.org/abs/2302.04761.

[9] Chengpeng Li, Mingfeng Xue, Zhenru Zhang, Jiaxi Yang, Beichen Zhang, Xiang Wang, Bowen Yu, Binyuan Hui, Junyang Lin, and Dayiheng Liu. Start: Self-taught reasoner with tools. *arXiv preprint arXiv:2503.04625*, 2025.

[10] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjun Zhong. Retool: Reinforcement learning for strategic tool use in llms, 2025. URL https://arxiv.org/abs/2504.11536.

[11] Guhao Feng, Bohang Zhang, Yuntian Gu, Haotian Ye, Di He, and Liwei Wang. Towards revealing the mystery behind chain of thought: a theoretical perspective. *Advances in Neural Information Processing Systems*, 36:70757–70798, 2023.

[12] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

[13] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.

[14] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. Critic: Large language models can self-correct with tool-interactive critiquing. *arXiv preprint arXiv:2305.11738*, 2023.

[15] Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. Limo: Less is more for reasoning. *arXiv preprint arXiv:2502.03387*, 2025.

[16] Trung Quoc Luong, Xinbo Zhang, Zhanming Jie, Peng Sun, Xiaoran Jin, and Hang Li. Reft: Reasoning with reinforced fine-tuning. *arXiv preprint arXiv:2401.08967*, 3, 2024.

[17] Richard Yuanzhe Pang, Weizhe Yuan, He He, Kyunghyun Cho, Sainbayar Sukhbaatar, and Jason Weston. Iterative reasoning preference optimization. *Advances in Neural Information Processing Systems*, 37:116617–116637, 2024.

[18] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*, 2023.

[19] Peiyi Wang, Lei Li, Zhihong Shao, RX Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. *arXiv preprint arXiv:2312.08935*, 2023.

[20] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[21] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[22] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems, 2024.

[23] Qwen Team. Qwq-32b: Embracing the power of reinforcement learning, March 2025. URL https://qwenlm.github.io/blog/qwq-32b/.

[24] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2, 2025. Notion Blog.